```
Ext.define('Hammerhead.grid.receiving.BoxQuantityGrid', {
    extend: 'Hammerhead.grid.GridField',
    alias: 'widget.boxquantitygrid',
    allowTypeSelection: true,
    allowIndicators: false,
    defaultType: 1,
    boxNumbersStore: Ext.create('Ext.data.ArrayStore', {
        fields: ['id'],
        data: []
    }),
    boxNum: 0,
    selModel: {
        selType: 'checkboxmodel',
        checkOnly: true,
        allowDeselect: true,
        mode: 'MULTI'
    },
    reqMark: '<span class="requiredMark">*</span>',
    viewConfig: {
        deferEmptyText: false,
        //CKS HAMMER-3758
        //emptyText: '<div class="x-grid-empty">No Box/Piece Items to Display</div>'
        emptyText: '<div class="x-grid-empty">Click \'Add\' to Associate Received Quan
tities with Available Boxes/Pieces</div>'
    },
    initComponent: function() {
        var me = this;
        me.setBoxNumbers();
        me.extraButtons = ['|',{
            text : 'Reprint Label',
            disabled: !me.received,
            tooltip : 'Reprint Label',
            handler : me.doAction,
            scope : me
        },{
                xtype: 'numberfield',
                width: 75,
                disabled: !me.received,
                name: 'copies',
        }, ' Copies','|',{
            text : 'Auto Create Boxes',
            disabled: me.received || me.store.getCount() > 0,
            tooltip : 'Auto Create Boxes and default quantity',
            handler : me.doAutoCreateBox,
            scope : me
        },{
                xtype: 'numberfield',
                width: 75,
                disabled: me.received || me.store.getCount() > 0,
                name: 'defaultqty',
        },{
                xtype: 'displayfield', //HAMMER-4468
                value: ' Default Quantity Per Box/Piece'
        } ];
        me.columns = [{
            header: me.reqMark + 'Box/Piece',
            dataIndex: 'boxnum',
            editable: false,
            editor: {
                xtype: 'combo',
                displayField: 'id',
                valueField: 'id',
                editable: false,
                store: me.boxNumbersStore,
                queryMode: 'local'
```

```
            }
        },{
            header: 'Quantity Per Box/Piece',
            dataIndex: 'quantity',
            editor: {
                xtype: 'numberfield',
                minValue: 0
            }

        }];
        me.callParent(arguments);
        me.getDockedComponent(1).getComponent(0).setDisabled(me.received && !me.existi
ngDiscrepant);
        me.getDockedComponent(1).getComponent(2).setDisabled(me.received && !me.existi
ngDiscrepant);
    },
    onCancelEdit: function(plugin, e) {
        var me = this;
        if(Ext.isEmpty(e.record.get('boxnum')) && Ext.isEmpty(e.record.get('quantity')
)) {
            me.store.remove(e.record);
        }
        if (me.received || me.store.getCount() > 0) {
                me.enableAutoCreate(true);
        } else {
                me.enableAutoCreate(false);
        }
    },
    onEdit: function(rowediting,e) {

    },
    onRemoveButton: function() {
        var me = this;
        me.callParent();
        if (me.received || me.store.getCount() > 0) {
                me.enableAutoCreate(true);
        } else {
                me.enableAutoCreate(false);
        }
    },
    onAddButton: function() {
        var me = this,rowEditor = me.getRowEditor(),recordCount = me.store.getCount(),
        newBoxnum=me.padToThree(me.setNextBoxNum());
        var r = Ext.create('Hammerhead.model.receiving.BoxItem', {
                r_lid: me.line_id,
                boxnum: null,
                quantity: null,
                id: recordCount++
        });
        me.store.add(r);
        rowEditor.startEdit(r, 0);
        if (me.received || me.store.getCount() > 0) {
                me.enableAutoCreate(true);
        } else {
                me.enableAutoCreate(false);
        }
    },
    enableAutoCreate: function(f) {
        var me = this;
        me.getDockedComponent(1).getComponent(8).setDisabled(f);
        me.getDockedComponent(1).getComponent(9).setDisabled(f);
        me.getDockedComponent(1).getComponent(10).setDisabled(f);
    },
    padToThree: function(number) {
        if (number <= 999) {
```

```
                number = ("00" + number).slice(-3);
        }
        return number;
    },
    setNextBoxNum: function() {
        var me = this;
        if (me.boxNum > 0){
                me.boxNum = me.boxNum+1;
        } else {
                me.boxNum = (me.store.initialBoxId)?me.store.initialBoxId:1;
        }
        return me.boxNum;
    },
    doAction: function() {
            var me = this,
            url = Hammerhead.data.BASE_URL,
            s = me.getSelectionModel().getSelection(),
            copies = me.query('toolbar')[0].child('numberfield').getValue(),
            printPLLabel = false,
            printParams = {}, qty = 0;
            if(Ext.isEmpty(s)) {
                Ext.Msg.alert('Warning', 'Please select a box/piece(s) to reprint.');
                return false;
            }
            if (me.checkValid()) {
            printParams.mask = new Ext.LoadMask({target:me, msg: 'Printing label...',
removeMask: true});
            printParams.mask.show();
            Ext.Array.each(s, function(r) {
                if (r.dirty) {
                        qty = r.get('quantity');
                }
                if (copies == null)
                        copies = 1;
//              if (Hammerhead.CurrentUser.printAsPDF()) {
                        url = Hammerhead.data.BASE_URL;
                        url += 'print/pdf/cpsLabel.rpt';
                        url += '?receiptId=0';
                        url += '&totalLabelCount=';
                        url += copies;
                        url += '&boxId=';
                        url += r.get('rbox_id');
                        url += '&printPLLabel=';
                        url += printPLLabel;
                        printParams.mask.hide();
                        window.open(url);
//              } else {
//                      Ext.Ajax.request({
//                          url: Hammerhead.data.BASE_URL + 'print/label/cpsLabel',
//                          method: 'POST',
//                          params: {boxId: r.get('rbox_id'), quantity: qty, receiptId:
null, totalLabelCount: copies != null ? copies : 1, isLocLabel: null, carrier: null, p
rintPLLabel: null},
//                          callback: function(options, success, response) {
//                              if (success && response.responseObject.labelPrintDataLis
t) {
//                                  printParams.content = response.responseObject.labelP
rintDataList.rptData;
//                                  Hammerhead.printing.LabelPrintingLib.print(printPara
ms);
//                              }
//                          },
//                          scope: me
//                      });
//              }
```

```
            });
            }


    },
    doAutoCreateBox: function() {
        var me = this, boxId = me.boxId !== 0 ? me.boxId : 1,
                boxMax = boxId + me.pieces.getValue(),
                defaultqty = me.query('toolbar')[0].getComponent(9).getValue(),
                rowEditor = me.getRowEditor(),recordCount = me.store.getCount(),
                newBoxnum=me.padToThree(me.setNextBoxNum());
        for (var i = boxId; i < boxMax; i++) {
                number = ("00" + i).slice(-3);
            me.store.add(Ext.create('Hammerhead.model.receiving.BoxItem', {
                r_lid: me.line_id,
                boxnum: number,
                quantity: defaultqty,
                id: recordCount++
            }));
                }
        if (me.received || me.store.getCount() > 0) {
                me.enableAutoCreate(true);
        } else {
                me.enableAutoCreate(false);
        }
    },
    checkValid:function(){
        var me = this, qpb_total=0;
        qpb_total=me.store.sum('quantity');
        if (qpb_total !== me.rec_qty) {
            Ext.Msg.alert('Warning', 'Total Quantity Per Box/Piece ('+qpb_total+') doe
s not equal the line item received quantity ('+me.rec_qty+')');
            return false;
        }
        return true;
    },
    setBoxNumbers: function() {
        var me = this, boxId = me.boxId !== 0 ? me.boxId : 1, boxMax = boxId + me.piec
es.getValue(); number = '';
        me.boxNumbersStore.removeAll(true);
        for (var i = boxId; i < boxMax; i++) {
                number = ("00" + i).slice(-3);
                me.boxNumbersStore.add({id: number});
                }
    }
});
```